

Yes, you absolutely can use Apache Kafka through **WSL 2 (Windows Subsystem for Linux)**, and it is **an excellent idea**. In fact, leading Kafka experts (including Confluent, the company behind commercial Kafka) highly recommend using WSL 2 or Docker over native Windows batch scripts for local development.

Why is WSL 2 a Better Idea Than Native Windows?

While native Windows scripts work for a quick 10-minute demo, running Kafka natively on Windows comes with severe limitations:

- **POSIX File System Dependencies:** Kafka is designed from the ground up for Linux. It relies heavily on POSIX features for memory-mapped files and high-throughput disk I/O.
- **The Infamous File Locking Bug:** On native Windows, Kafka often throws fatal exceptions when trying to clean up or roll log segments because Windows locks open files differently than Linux. This frequently results in brokers crashing randomly during local development.
- **Topic Deletion Issues:** Deleting a topic natively on Windows often fails or leaves the file system in a corrupted state.
- **Production Parity:** Production Kafka clusters are virtually always deployed on Linux. Developing inside WSL 2 ensures your development environment identically mirrors how the code will act in production.

How to Set Up Kafka on WSL 2 (KRaft Mode)

Here is how to get it running inside your WSL terminal (assuming you are using a standard distribution like Ubuntu).

1. Install Java Inside WSL

Open your WSL terminal and update your package manager, then install Java:

```
Bash
sudo apt update
sudo apt install default-jdk -y
```

Verify it works: `java -version`

2. Download and Extract Kafka

Download the standard Linux binary archive directly into your WSL environment:

```
Bash
wget https://archive.apache.org/dist/kafka/3.9.0/kafka_2.13-3.9.0.tgz
tar -xzf kafka_2.13-3.9.0.tgz
cd kafka_2.13-3.9.0
```

3. Format Storage and Start Kafka (KRaft Mode)

Generate a cluster UUID:

Bash

```
./bin/kafka-storage.sh random-uuid
```

(Copy the string outputted, e.g., `d6F4s...`)

Format the directory using that UUID:

Bash

```
./bin/kafka-storage.sh format -t cXRpGhH_QgyWnByTNus4Yw -c ./config/kraft/server.properties
```

Start the server:

Bash

```
./bin/kafka-server-start.sh ./config/kraft/server.properties
```



The One Gotcha: Connecting from Windows to WSL 2

If your application code (e.g., a Spring Boot app, Python script, or C# code) is running inside **Windows IDEs** (like IntelliJ or VS Code on the Windows host) but your Kafka broker is running inside **WSL 2**, they live on slightly different network layers.

By default, WSL 2 maps `localhost:9092`, but Kafka's internal metadata mechanism might broadcast a different address, causing connection timeouts like `"Broker may not be available"`.

The Fix:

Before starting your Kafka server in WSL, edit your `config/kraft/server.properties` file:

1. Locate the `advertised.listeners` line.
2. Uncomment it or update it to explicitly look like this:
3. Plaintext

```
advertised.listeners=PLAINTEXT://localhost:9092
```

- 4.
- 5.

This tells Kafka to advertise itself as `localhost`, allowing your host Windows applications to easily pass data through the WSL network bridge seamlessly.

Here is a clean, professionally formatted version of the instructions designed to be pasted directly into Google Docs. It uses standard Markdown elements that translate perfectly into Google Docs' rich text formatting.

How to Fix Kafka "Connection Refused" Error (Port 2181)

The error `java.net.ConnectException: Connection refused` on port 2181 means **ZooKeeper is not running or is not configured correctly**.

Kafka (especially older versions or those using the legacy ZooKeeper mode) **cannot start** without a running ZooKeeper instance. The logs show Kafka trying to connect to `localhost:2181` and failing because nothing is listening on that port.

Method 1: Start ZooKeeper First (Recommended Quick Fix)

You must start ZooKeeper **before** starting Kafka.

Step 1: Start ZooKeeper

1. **Navigate to your Kafka directory** in WSL:

2. Bash

```
cd ~/kafka_2.13-3.x.x # Adjust path if your version is different
```

3.

4.

5. **Start ZooKeeper** using the default configuration:

6. Bash

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

7.

8.

9. **Wait for the logs** to display lines indicating success, such as: `Started AdminServer on address...` and binding to port 2181.

10. **Keep this terminal window open** (do not close it or stop the process).

Step 2: Start Kafka

1. Open a **new terminal window** (leave the ZooKeeper terminal running).
2. Navigate to your Kafka directory again.
3. Start the Kafka broker:

4. Bash

```
bin/kafka-server-start.sh config/server.properties
```

5.

6.

Kafka should now successfully connect to ZooKeeper on `localhost:2181`.

Quick Summary of Commands:

Terminal 1 (ZooKeeper):

```
bin/zookeeper-server-start.sh config/zookeeper.properties
```

Terminal 2 (Kafka):

```
bin/kafka-server-start.sh config/server.properties
```

Here is the formatted version of your next steps, ready to be pasted cleanly into your Google Docs workflow document:

Testing the Kafka Connection from Windows

After configuring Kafka to listen properly across the WSL boundary, use the following steps to verify the connection and update your application configuration.

1. Quick Connection Test (PowerShell)

Run this command in a standard **Windows PowerShell** window (Administrator privileges are not required):

```
PowerShell
Test-NetConnection -ComputerName localhost -Port 9092
```

Expected Results:

- `TcpTestSucceeded : True` $\rightarrow$ **Success!** Kafka is running and reachable from Windows.
- `TcpTestSucceeded : False` $\rightarrow$ **Connection Failed.** Re-check your `advertised.listeners` configuration inside WSL.

Integration started below

2. Update Application Configuration

You can now point your Windows-based applications directly to `localhost`. You do **not** need to hunt down the dynamic WSL IP address.

Update your application properties (e.g., Spring Boot `application.properties`, Python scripts, or Node.js configurations) to the following:

```
Properties
bootstrap.servers localhost:9092
```

3. Verify with a End-to-End Topic Test (Recommended)

To ensure messages are actively flowing between Windows and your WSL Kafka broker, perform a quick smoke test:

Step A: Create a Test Topic (Inside WSL)

Run the following command in your WSL terminal to spin up a new topic named `demo-topic`:

Bash

```
bin/kafka-topics.sh --create --topic demo-topic --bootstrap-server localhost:9092 --partitions 1
--replication-factor 1
```

Step B: Produce/Consume from Windows

Run your application or an external tool (like Kafkahq, Offset Explorer, or a basic script) from the Windows environment to send a message to `demo-topic`.

Note: If your application encounters connection timeouts or data serialization errors during this test, copy and paste the stack trace here for troubleshooting!

You need to create the topic before running your producer.

Option 1: Create the topic manually (Recommended)

Run this command in your WSL terminal (where Kafka is running):

bash

```
bin/kafka-topics.sh --create --topic <YOUR_TOPIC_NAME>
--bootstrap-server localhost:9092 --partitions 1
--replication-factor 1
```

Replace `<YOUR_TOPIC_NAME>` with the exact topic name used in your Go code (e.g., `demo-topic` `test`, etc.).